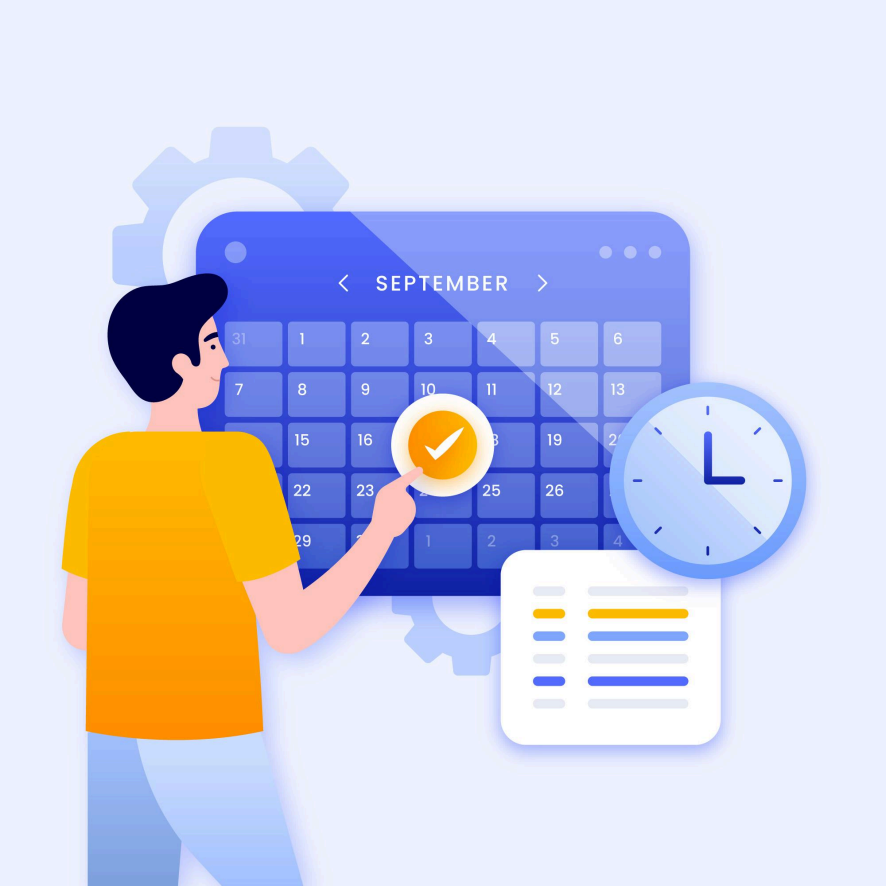
salesianos TRIANA | Colegio San Pedro



REGISTRO DE JORNADA

TIPO C

PROYECTO DE DESARROLLO DE UNA API REST CON SPRING BOOT

ACCESO A DATOS
PROGRAMACIÓN DE SERVICIOS Y PROCESOS
CURSO 2026-27

INTRODUCCIÓN

El proyecto consiste en la implementación de una API REST que permita gestionar información sobre el registro de jornada de una empresa, incluyendo:

- **Las peticiones que se detallan a continuación.**
- **La documentación generada con OpenApi 3.0 y Swagger**
- **Una colección de Postman consistente para poder probar la aplicación.**
- **Una documentación elaborada con Markdown para incluir como README.md del repositorio.**

MODELO DE DATOS

Este proyecto simula un sistema de control horario, con **tres entidades** y una relación de **uno a muchos** (Un Empleado tiene muchos Fichajes).

Entidades a Modelar

1. **Departamento** (Entidad Principal):
   - id (Long, PK).
   - nombre (String, único).
   - presupuesto (BigDecimal).
2. **Empleado** (Entidad Secundaria):
   - id (Long, PK).
   - nombreCompleto (String).
   - cargo (String).
   - salario (BigDecimal).
   - **Asociación:** Un Departamento tiene muchos Empleados.
3. **Fichaje** (Entidad de Asociación/Registro):
   - id (Long, PK).
   - momento (LocalDateTime): Registro de la hora de entrada o salida.
   - tipo (Enum: ENTRADA, SALIDA).
   - **Asociación:** Un Empleado tiene muchos Fichajes.

Requisitos Funcionales Clave

- **Departamentos y Empleados:** CRUD completo para ambas entidades.
- **Asociación:**
  - `PUT /api/v1/empleados/{empleadoId}/departamento/{deptoId}`: Asignar un empleado a un departamento existente.
- **Fichajes (Registros):**
  - `POST /api/v1/empleados/{empleadoId}/fichar`: Registrar una nueva **ENTRADA** o **SALIDA** para un empleado. La lógica de negocio debe

alternar automáticamente entre ENTRADA y SALIDA.
    - `GET /api/v1/empleados/{empleadoId}/fichajes`: Obtener el historial de fichajes de un empleado.

Validación de entradas, Gestión de Errores y Calidad

- Utilizar todas las anotaciones de validación necesarias para que la lógica de proyecto sea correcta.
- También se debe incluir una anotación de validación personalizada, que también sea acorde a la lógica de negocio. Por ejemplo, si la suma de los salarios de un departamento excede su presupuesto.

- **Excepciones Requeridas (Mapeadas a ProblemDetail):**
    - `XXXXNoEncontradaException` (404 Not Found) para Departamento o Empleado.
    - `FichajeDuplicadoException` (409 Conflict): Lanzada si se intenta registrar una ENTRADA cuando el último fichaje registrado ya fue una ENTRADA (o una SALIDA cuando el último fue una SALIDA).
- **Diseño:** Uso obligatorio de **DTOs** (ej: EmpleadoResponseDTO debe incluir el nombre del Departamento). Si es necesario, uso de DTOs anidados (un DTO que tiene como campo otro DTO).
- **Documentación:** todos los endpoints tal y como hemos descrito en clase.

METODOLOGÍA, ENTREGA y EVALUACIÓN

- El proyecto se realizará individualmente.
- Se recomienda utilizar una *versión express* de una metodología ágil, tratando de
    - Trabajar por historias de usuario, requisitos o especificaciones.
    - La entrega del proyecto se realizará a través de un repositorio de código, adjuntando la URL del mismo en el punto de entrega Google Classroom. Inicialmente será privado. **Una vez que la url del repositorio esté entregada, el repositorio debe tener acceso público. Si no lo tiene, no constará como entregado.**
- Para evaluar el proyecto, se utilizará una rúbrica con los referentes de evaluación de la unidad didáctica (publicados en Classroom)

Se estipula como fecha de entrega el **viernes 23 de Octubre antes de las 14:30**

En el repositorio se debe incluir:

- Todo el código fuente del proyecto
- La colección de postman exportada con las peticiones para probar el API

- Un fichero README.md (curradito, por favor) con la documentación del proyecto, como si lo tuviera que leer cualquier persona externa.
-